

QUESTION NO#01:

```
#include <iostream>

#include <string>

using namespace std;

class student {
public:
    string name;
    int roll_no;
    int semester;
    char section;
};

int main() {
    // Creating 4 student objects
    student s1 = {"Ali", 101, 2, 'A'};
    student s2 = {"Sara", 102, 2, 'B'};
    student s3 = {"Ahmed", 103, 3, 'A'};
    student s4 = {"Zara", 104, 1, 'C'};

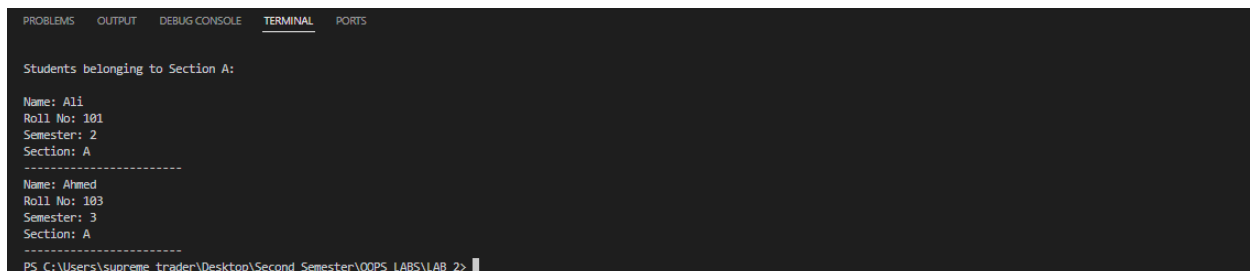
    student students[4] = {s1, s2, s3, s4};

    cout << "Students belonging to Section A:\n\n";

    for (int i = 0; i < 4; i++) {
```

```
if (students[i].section == 'A') {  
    cout << "Name: " << students[i].name << endl;  
    cout << "Roll No: " << students[i].roll_no << endl;  
    cout << "Semester: " << students[i].semester << endl;  
    cout << "Section: " << students[i].section << endl;  
    cout << "-----\n";  
}  
}  
  
}
```

OUTPUT:



The screenshot shows a terminal window with the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
Students belonging to Section A:  
  
Name: Ali  
Roll No: 101  
Semester: 2  
Section: A  
-----  
Name: Ahmed  
Roll No: 103  
Semester: 3  
Section: A  
-----  
PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 2>
```

QUESTION NO#02:

```
#include <iostream>

using namespace std;

class BankAccount {
private:
    double balance;
    int transactions;
    double interestRate;

public:

    BankAccount(double initialBalance = 0.0) {
        balance = initialBalance;
        transactions = 0;
        interestRate = 0.05; // 5% interest rate
    }

    void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            transactions++;
            cout << "Deposit successful.\n";
        } else {
            cout << "Invalid deposit amount.\n";
        }
    }
};
```

```
    }  
}  
  
void withdraw(double amount) {  
    if (amount > 0 && amount <= balance) {  
        balance -= amount;  
        transactions++;  
        cout << "Withdrawal successful.\n";  
    } else {  
        cout << "Invalid or insufficient balance.\n";  
    }  
}  
  
void displayBalance() {  
    cout << "Current Balance: $" << balance << endl;  
}  
  
void displayTransactions() {  
    cout << "Number of Transactions: " << transactions << endl;  
}  
  
void displayInterest() {  
    double interest = balance * interestRate;  
    cout << "Interest Earned: $" << interest << endl;  
}  
};
```

```
int main() {  
    BankAccount account(1000);  
  
    int choice;  
  
    double amount;  
  
    do {  
        cout << "\n--- ABC Bank Menu ---\n";  
        cout << "1. Display the account balance\n";  
        cout << "2. Display the number of transactions\n";  
        cout << "3. Display interest earned for this period\n";  
        cout << "4. Make a deposit\n";  
        cout << "5. Make a withdrawal\n";  
        cout << "6. Exit the program\n";  
        cout << "Enter your choice: ";  
        cin >> choice;  
  
        switch (choice) {  
            case 1:  
                account.displayBalance();  
                break;  
  
            case 2:  
                account.displayTransactions();  
                break;
```

case 3:

```
account.displayInterest();
```

```
break;
```

case 4:

```
cout << "Enter deposit amount: ";
```

```
cin >> amount;
```

```
account.deposit(amount);
```

```
break;
```

case 5:

```
cout << "Enter withdrawal amount: ";
```

```
cin >> amount;
```

```
account.withdraw(amount);
```

```
break;
```

case 6:

```
cout << "Thank you for using ABC Bank.\n";
```

```
break;
```

default:

```
cout << "Invalid choice. Try again.\n";
```

```
}
```

```
} while (choice != 6);
```

```
    return 0;  
}
```

OUTPUT:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
--- ABC Bank Menu ---  
1. Display the account balance  
2. Display the number of transactions  
3. Display interest earned for this period  
4. Make a deposit  
5. Make a withdrawal  
6. Exit the program  
Enter your choice: 3  
Interest Earned: $50
```

```
--- ABC Bank Menu ---  
1. Display the account balance  
2. Display the number of transactions  
3. Display interest earned for this period  
4. Make a deposit  
5. Make a withdrawal  
6. Exit the program  
Enter your choice: 2  
Number of Transactions: 0
```

```
--- ABC Bank Menu ---  
1. Display the account balance  
2. Display the number of transactions  
3. Display interest earned for this period  
4. Make a deposit  
5. Make a withdrawal  
6. Exit the program  
Enter your choice: 5  
Enter withdrawal amount: 2000  
Invalid or insufficient balance.
```

```
--- ABC Bank Menu ---  
1. Display the account balance  
2. Display the number of transactions  
3. Display interest earned for this period  
4. Make a deposit  
5. Make a withdrawal  
6. Exit the program  
Enter your choice: 6  
Thank you for using ABC Bank.
```

PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 2>

QUESTION NO#03:

```
#include <iostream>

#include <cstring>

#include <string>

using namespace std;

class Employee {

private:

    char* firstName;  // Dynamic Memory Allocation

    string lastName;

    int monthlySalary;

public:

    // Default Constructor

    Employee() {

        firstName = nullptr;

        monthlySalary = 0;

    }

    // Parameterized Setter

    void setEmployee(const char* fName, string lName, int salary) {

        // Delete old memory if any

        delete[] firstName;

        // Allocate memory for first name
```



```
    firstName = new char[strlen(fName) + 1];
    strcpy(firstName, fName);

    lastName = lName;

    if (salary > 0)
        monthlySalary = salary;
    else
        monthlySalary = 0;
}

// Copy Constructor (Rule of Three)
Employee(const Employee& obj) {
    firstName = new char[strlen(obj.firstName) + 1];
    strcpy(firstName, obj.firstName);
    lastName = obj.lastName;
    monthlySalary = obj.monthlySalary;
}

// Copy Assignment Operator (Rule of Three)
Employee& operator=(const Employee& obj) {
    if (this != &obj) {
        delete[] firstName;

        firstName = new char[strlen(obj.firstName) + 1];
        strcpy(firstName, obj.firstName);
```

```
        lastName = obj.lastName;
        monthlySalary = obj.monthlySalary;
    }
    return *this;
}
```

// Getter Functions

```
char* getFirstName() const {
    return firstName;
}
```

```
string getLastName() const {
    return lastName;
}
```

```
int getMonthlySalary() const {
    return monthlySalary;
}
```

// Calculate Yearly Salary

```
int getYearlySalary() const {
    return monthlySalary * 12;
}
```

// Give 10% Raise

```
void giveRaise() {
    monthlySalary += monthlySalary * 10 / 100;
}

// Destructor (Rule of Three)
~Employee() {
    delete[] firstName;
}

};

int main() {
    Employee emp1, emp2;

    // Initialize employees
    emp1.setEmployee("Alisha", "Khan", 50000);
    emp2.setEmployee("Sana", "Ali", 60000);

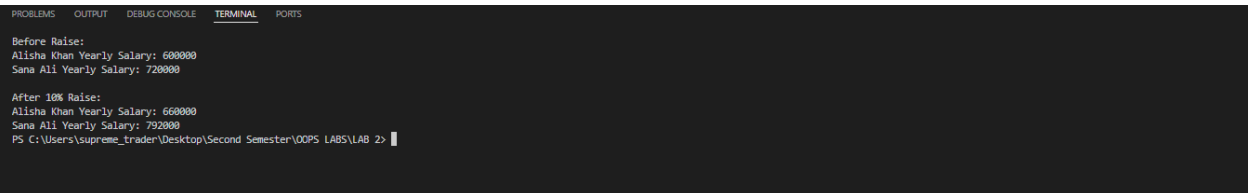
    // Display yearly salary before raise
    cout << "Before Raise:\n";
    cout << emp1.getFirstName() << " " << emp1.getLastName()
        << " Yearly Salary: " << emp1.getYearlySalary() << endl;

    cout << emp2.getFirstName() << " " << emp2.getLastName()
        << " Yearly Salary: " << emp2.getYearlySalary() << endl;

    // Give 10% raise
```

```
emp1.giveRaise();  
emp2.giveRaise();  
  
// Display yearly salary after raise  
cout << "\nAfter 10% Raise:\n";  
cout << emp1.getFirstName() << " " << emp1.getLastName()  
    << " Yearly Salary: " << emp1.getYearlySalary() << endl;  
  
cout << emp2.getFirstName() << " " << emp2.getLastName()  
    << " Yearly Salary: " << emp2.getYearlySalary() << endl;  
  
return 0;  
}
```

OUTPUT:



The screenshot shows a terminal window with the following output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
Before Raise:  
Alisha Khan Yearly Salary: 600000  
Sana Ali Yearly Salary: 720000  
  
After 10% Raise:  
Alisha Khan Yearly Salary: 660000  
Sana Ali Yearly Salary: 792000  
PS C:\Users\Supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 2> |
```

QUESTION NO#04:

```
#include <iostream>

#include <cstdlib> // Required for rand() and srand()

#include <ctime> // Required for time()


using namespace std;


class HittingGame {
private:
    int totalPlayers;
    int myTeamScore;
    int enemyTeamScore;

public:
    // Constructor to initialize the game state
    HittingGame() {
        myTeamScore = 0;
        enemyTeamScore = 0;

        // Randomly generate total players (Let's stick to a range of 1-5 for simplicity)
        totalPlayers = (rand() % 5) + 1;
    }
}
```

```
// Main function to run the game logic

void play() {

    cout << "Total No. Of Players in your team: " << totalPlayers << endl << endl;

    for (int i = 0; i < totalPlayers; i++) {

        cout << "Pair of numbers:" << endl;

        // Generate pairs of numbers

        // Using a small range (1-5) so matches happen frequently,
        // similar to the screenshot logic.

        int number1 = (rand() % 5) + 1;

        int number2 = (rand() % 5) + 1;

        cout << "Number1: " << number1 << endl;

        cout << "Number2: " << number2 << endl;

        // Logic: If numbers match, you hit them. If not, they hit you.

        if (number1 == number2) {

            cout << "Enemy got hit by your team!" << endl;

            myTeamScore++;

        } else {

            cout << "You got hit by the enemy team!" << endl;

            enemyTeamScore++;

        }

        cout << endl; // Empty line for formatting

    }

}
```

```
        showFinalResult();
    }

    // Helper function to display the winner
    void showFinalResult() {
        if (myTeamScore > enemyTeamScore) {
            cout << "Game Over! You won" << endl;
        } else if (enemyTeamScore > myTeamScore) {
            cout << "Game Over! Enemy won" << endl;
        } else {
            cout << "Game Over! It's a Draw" << endl;
        }
    }
};

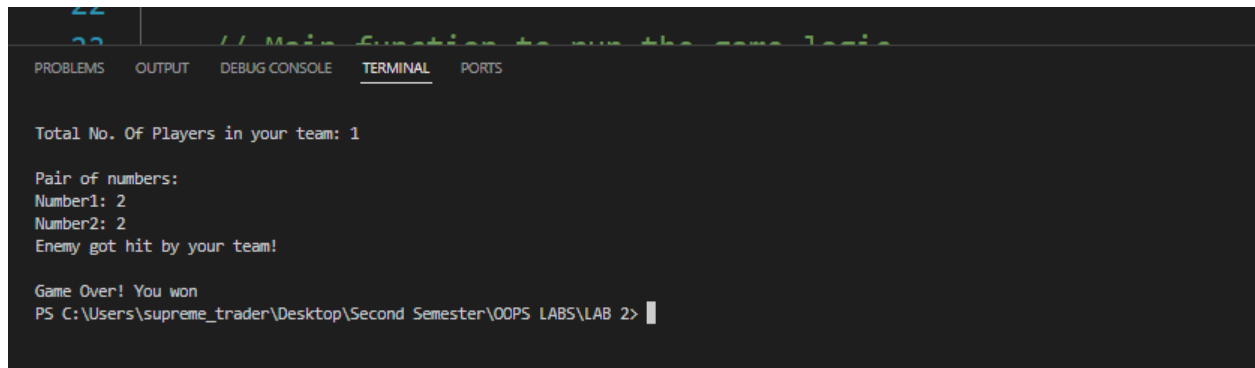
int main() {
    // Seed the random number generator so numbers change every time the program runs
    srand(time(0));

    // Create the game object
    HittingGame game;

    // Start the game
    game.play();
```

```
    return 0;  
}
```

OUTPUT:



The screenshot shows a C++ IDE with a terminal window open. The terminal displays the following output:

```
// Main function to run the game logic  
  
Total No. Of Players in your team: 1  
  
Pair of numbers:  
Number1: 2  
Number2: 2  
Enemy got hit by your team!  
  
Game Over! You won  
PS C:\Users\supreme_trader\Desktop\Second Semester\OOPS LABS\LAB 2>
```